



Università
di Catania

UNIVERSITÀ DEGLI STUDI DI CATANIA

Dipartimento di Matematica e Informatica

Corso di Laurea Magistrale in Informatica (LM-18)

RELAZIONE DI PROGETTO

Sistemi Cloud

PayMySeat

Gestione della transazionalità economica e della coerenza
in un'architettura cloud-native a microservizi

Docenti

Prof. Giuseppe Pappalardo

Prof. Salvatore Nicotra

Candidata

Valeria Platania

Matricola 1000081895

Anno Accademico 2025/2026

<https://github.com/ValeriaPlatania/PayMySeat>

Indice

1	Introduzione	3
1.1	Il contesto e il problema del dominio	3
1.2	Obiettivi primari e secondari del progetto	4
2	Architettura e Pattern Architetture	5
2.1	Visione d'insieme e scomposizione dei microservizi	5
2.2	La scelta della persistenza poliglotta	6
2.3	Il Pattern Transactional Outbox (Spiegazione dettagliata)	6
2.4	Il Pattern Saga e la Coreografia per i rimborsi	8
3	L'Idempotenza e la gestione della Concorrenza	9
3.1	Concetto teorico e utilità dell'Idempotenza	9
3.2	Primo scudo protettivo: La Cache Redis ad alte prestazioni	9
3.3	Secondo scudo protettivo: I vincoli di unicità nel database Relazionale	10
4	Sicurezza Logica e Webhook Crittografici	11
4.1	Il vettore d'attacco dello Spoofing	11
4.2	Prevenzione mediante firma crittografica HMAC	11
5	Containerizzazione e Infrastruttura di Esecuzione	13
5.1	Riproducibilità totale tramite Docker	13
5.2	Orchestrazione locale con Docker Compose	14
6	Infrastructure as Code e Distribuzione su AWS Cloud	15
6.1	La Pipeline di Provisioning: HashiCorp Terraform	15
6.2	La Pipeline di Configurazione software: Ansible	16
7	Validazione Sperimentale e Sperimentazione Empirica	17
7.1	Verifica intensiva dell'Idempotenza API sotto Race Condition	17
7.2	Verifica estrema della Resilienza Outbox via Chaos Engineering	18
8	Conclusioni	19
.1	Evoluzione dalla Proposta al MVP (Minimum Viable Product)	19

A	Manuale d’Uso e Replicazione delle Prove	21
A.1	Prerequisiti di Sistema	21
A.2	Avvio dell’Infrastruttura	21
A.3	Prova 1: Flusso di Pagamento Standard (Happy Path)	22
A.4	Prova 2: Test Concorrenziale di Idempotenza	22
A.5	Prova 3: Test di Resilienza (Chaos Engineering)	23
A.6	Prova 4: Test di Compensazione (Saga Pattern)	23
A.7	Prova 5: Deploy in Cloud (AWS)	24

Capitolo 1

Introduzione

1.1 Il contesto e il problema del dominio

Nel contesto dei sistemi distribuiti su larga scala, la gestione delle transazioni economiche presenta sfide architetturali non trascurabili. In un ecosistema a microservizi in cui le responsabilità sono suddivise tra molteplici attori, garantire l'atomicità, la consistenza, l'isolamento e la durabilità (le canoniche proprietà ACID dei database relazionali) su database separati fisicamente e logicamente è impossibile senza ricorrere a complessi protocolli di coordinamento distribuito. Protocolli come il *Two-Phase Commit* (2PC), tuttavia, introducono latenze inaccettabili e single point of failure (SPOF), andando diametralmente contro i principi del paradigma *Cloud-Native*, che predilige la disponibilità e la partizionabilità (secondo il ben noto Teorema CAP).

Il progetto **PayMySeat** nasce per affrontare specificamente e puntualmente questo problema in collaborazione (e parziale contrapposizione) con il sistema di biglietteria **HoldMySeat**. Mentre HoldMySeat si occupa della complessa gestione della concorrenza sui posti fisici all'interno di una mappa (gestendo le prenotazioni per risorse strettamente non replicabili), PayMySeat opera come un gateway di pagamento asincrono e autonomo, un vero e proprio sistema bancario simulato ma architetturalmente reale.

Il problema fondamentale che è scaturito da questa interazione è il seguente: quando un utente acquista un posto, il pagamento deve essere addebitato una volta sola, e il sistema di biglietteria deve essere notificato con l'assoluta certezza dell'avvenuto incasso. Se si verifica un timeout di rete (che impedisce alla risposta HTTP di tornare al client), se l'utente aggiorna compulsivamente la pagina in attesa di un feedback visivo, o se un servizio si riavvia improvvisamente (causa scaling autogestito da un orchestratore come Kubernetes o un blocco dell'interprete), si potrebbe facilmente verificare un doppio addebito (perdita economica per il cliente e conseguente danno d'immagine e potenziale frode) o, viceversa, un addebito senza l'emissione effettiva del biglietto (perdita del servizio erogato).

La *coerenza distribuita* diventa quindi il nucleo centrale attorno a cui ruota l'intero ciclo di vita del progetto. Occorre passare da un obsoleto modello di transazionalità forte e bloccante a un più resiliente modello di *Eventual Consistency* (coerenza eventuale), supportato da pattern architetturali robusti in grado di dirimere le inevitabili race conditions imposte dalla concorrenza asincrona.

1.2 Obiettivi primari e secondari del progetto

Il progetto persegue molteplici macro-obiettivi tecnici e metodologici, strutturati per fornire un prodotto pronto per la produzione. Tali obiettivi vengono esplicitati qui di seguito in ordine di priorità architetturale:

1. **Idempotenza transazionale:** Questa proprietà è essenziale per qualsiasi sistema finanziario. Il sistema deve essere in grado di ricevere molteplici richieste identiche per lo stesso ordine (identificato in modo univoco), e deve garantire che, indipendentemente da quante volte la richiesta venga processata in parallelo da nodi distribuiti, l'effetto sul database e sulle finanze dell'utente sia esattamente equivalente a quello di una singola e isolata richiesta.
2. **Resilienza e gestione dei guasti (Fault Tolerance):** L'assenza di single point of failure (SPOF) è un mandato fondamentale per i sistemi cloud-native. PayMySeat deve potersi riprendere da un crash senza l'intervento umano. Se il message broker (RabbitMQ) cade o diviene inaccessibile per un taglio di rete, i pagamenti appena processati non devono andare persi né restare in uno stato inconsistente. Se l'API cade durante la notifica al sistema esterno, le notifiche devono comunque essere inviate al ripristino. Per fare ciò, il progetto implementa rigorosamente il pattern *Transactional Outbox*.
3. **Automazione totale del ciclo di vita (Infrastructure as Code - IaC):** L'era in cui un sistemista entrava via SSH in una macchina per installare pacchetti a mano è tramontata. L'obiettivo qui è poter costruire e distruggere l'intero ambiente di esecuzione, partendo dalle reti virtuali (VPC) fino ai cluster Docker, passando per le regole firewall (Security Groups e NACL), attraverso codice dichiarativo e versionato. Non devono esistere server "pets" (creati, battezzati e curati a mano), ma solo server "cattle" (sostituibili, numerati e identici).
4. **Disaccoppiamento, incapsulamento e scalabilità orizzontale:** PayMySeat è un dominio a sé stante. Non deve, in nessun caso, condividere lo stesso database logico o fisico né la stessa area di memoria con HoldMySeat. Le comunicazioni devono avvenire scavalcando confini di rete rigidi. I comandi di pagamento o rimborso avvengono tramite chiamate HTTP RESTful sincrone, mentre le notifiche di esito viaggiano come eventi asincroni tramite Webhook. Questo garantisce che i due team di sviluppo possano rilasciare versioni, eseguire manutenzioni e scalare orizzontalmente in modo completamente asincrono e indipendente, preservando la pura architettura a microservizi.

Capitolo 2

Architettura e Pattern Architetturali

Questo capitolo illustra nel dettaglio l'infrastruttura logica pensata per assolvere ai requisiti definiti precedentemente, soffermandosi in particolare sui concetti chiave del Domain-Driven Design e della messaggistica asincrona.

2.1 Visione d'insieme e scomposizione dei microservizi

L'architettura di PayMySeat si basa sul paradigma dei microservizi orientati agli eventi (*Event-Driven Microservices* o EDA). La suddivisione delle responsabilità è stata effettuata seguendo i principi del *Domain-Driven Design* (DDD), definendo in modo inequivocabile i Bounded Contexts. Il dominio di PayMySeat comprende le entità core di Pagamento (Payment), Rimborso (Refund) ed Evento tracciabile (Event).

La topologia di deployment e di interazione a runtime è composta dai seguenti componenti principali:

- **Payment API:** Rappresenta il cuore sincrono e front-facing del sistema. Espone gli endpoint HTTP POST consumati dal client. È l'attore incaricato di validare il lock di idempotenza e di orchestrare l'inserimento sul database relazionale primario garantendone le proprietà ACID.
- **Outbox Relay:** Un demone in background (poller/worker) che agisce come collante tra il mondo sincrono del database e quello asincrono della messaggistica, rendendo possibile una comunicazione affidabile. Esegue query continue (polling) per recuperare messaggi appena scritti e li instrada al message broker.
- **Refund Worker:** Un worker puro, che si comporta da consumer asincrono per l'infrastruttura. Reagisce ai messaggi di cancellazione o timeout delle prenotazioni presenti nel broker ed esegue l'operazione di compensazione (rimborso), interfacciandosi in modo sicuro con le API bancarie di terze parti.
- **Gateway Adapter:** Questo componente agisce sia da *Anti-Corruption Layer* che da *Webhook dispatcher*. Si interpone fingendo di essere il provider bancario esterno (ad esempio Stripe o PayPal) e assume l'importante responsabilità di generare e applicare

le firme crittografiche (signatures) per la chiamata asincrona di callback (Webhook) verso HoldMySeat.

2.2 La scelta della persistenza poliglotta

Nell'ambito dei database, "one size fits all" è un antipattern in ambito Cloud-Native. Costringere tipi di dato diametralmente opposti nello stesso datastore porta inevitabilmente a compromessi inaccettabili in termini di performance. PayMySeat sposa il paradigma della persistenza poliglotta:

- **MySQL (Relational Database):** Lo store relazionale è stato scelto come *Single Source of Truth* per il dominio economico. La struttura delle tabelle, segnatamente quelle dei Pagamenti e della Outbox, è rigidamente normalizzata per evitare anomalie di inserimento, duplicazione e cancellazione. In un dominio finanziario, le foreign keys e i vincoli di consistenza (come i vincoli **UNIQUE**) del motore InnoDB sono l'unica linea di difesa invalicabile per il salvataggio effettivo del denaro.
- **Redis (In-Memory Key-Value Store):** Scelto esclusivamente per le sue incredibili performance sub-millisecondo in operazioni di lock. Redis implementa nativamente comandi atomici come **SETNX** (Set if Not eXists) e semantiche di scadenza autonoma dei record (TTL - Time To Live). Questa combinazione è il tassello perfetto per costruire un sistema di validazione preliminare dell'idempotenza, riducendo quasi a zero il carico su MySQL derivato da flood di richieste duplicate (ad esempio per l'esaurimento dei click di un utente spazientito).
- **MongoDB (NoSQL Document Store):** Svolge il ruolo cruciale di archivio storico o di *Audit Trail*. Nel corso del tempo, la struttura dei pagamenti e dei log può evolvere. MongoDB archivia ricevute e log transazionali registrando documenti JSON eterogenei e profondamente nidificati. In caso di dispute legali, failure analysis o scopi di conformità (compliance), MongoDB detiene la storia immutabile di ogni transito attraverso il broker di messaggi. In cloud, il suo diretto equivalente nativo è Amazon DocumentDB.

2.3 Il Pattern Transactional Outbox (Spiegazione dettagliata)

In un'architettura a microservizi asincroni, una delle azioni in assoluto più comuni e al contempo complesse è l'aggiornamento simultaneo di uno stato locale su database accoppiato alla pubblicazione di un evento verso il resto del mondo.

Prendiamo l'esempio vitale di PayMySeat. Quando la *Payment API* processa un ordine, la sua responsabilità logica si articola in due step:

1. Inserire o aggiornare il record con status **PAID** nel proprio database relazionale MySQL.
2. Segnalare a tutti gli altri servizi e subscriber (in particolare a HoldMySeat) che il pagamento si è concluso inviando un evento **PaymentSucceeded** su RabbitMQ.

Poiché il commit su MySQL e la pubblicazione su RabbitMQ sono operazioni afferenti a due server fisici distinti, dotati di driver e protocolli dissimili (SQL vs AMQP), il sistema è soggetto al classico problema della **dual-write**. Se il passo 1 ha pieno successo e i dati vengono committati, ma il passo 2 fallisce perché il broker è in manutenzione o ha la RAM saturata, il pagamento è stato correttamente salvato ed esatto dal conto del cliente, ma il sistema HoldMySeat non verrà mai notificato, non emetterà il biglietto, e lascerà la risorsa appesa e irraggiungibile per sempre (o peggio, il timeout locale della prenotazione libererà il posto rendendolo acquistabile da qualcun altro, generando overbooking o furto manifesto). Al contrario, se lo sviluppatore invertisse l'ordine (prima scrivo sul broker, poi sul DB), e il DB rifiutasse il commit per un deadlock, il broker emetterebbe un successo inesistente, e HoldMySeat cederebbe gratis il prezioso biglietto.

Soluzione adottata: Il Transactional Outbox pattern. Questo elegantissimo e consolidato pattern elimina il dual-write delegando il problema alle garanzie transazionali del singolo database relazionale.

L'inserimento del record del pagamento e l'inserimento dell'evento avvengono contestualmente all'interno della medesima transazione locale (`BEGIN TRANSACTION ... COMMIT`) in MySQL.

La tabella aggiuntiva, che chiamiamo `outbox_events`, agisce come coda temporanea. Se la transazione dovesse fallire per qualsivoglia motivo (timeout del lock, constraint violati), anche l'evento scartato subirebbe un `ROLLBACK` scomparendo nel nulla. Se la transazione ha successo, il pagamento e la volontà di spedire un messaggio diventano persistenti, immutabili e resistenti anche a disconnessioni di corrente del datacenter.

```
1 BEGIN TRANSACTION ;
2   -- Il record di dominio
3   INSERT INTO payments (id, amount, status)
4   VALUES ('PAY-123', 5000, 'COMPLETED');
5
6   -- L'intenzione di evento da comunicare
7   INSERT INTO outbox_events (event_type, payload, status)
8   VALUES ('payment.succeeded', '{"payment_id":"PAY-123"...}', 'PENDING');
9 COMMIT;
```

Listing 2.1: Rappresentazione pseudo-codice del Transactional Outbox

Un servizio totalmente separato dal thread di ricezione API, denominato *Outbox Relay* (o Message Relay), si avvia in esecuzione perenne. Il suo unico scopo è eseguire un polling continuo (o, in architetture più evolute, un listening sul transaction log - *CDC*): recupera gli eventi marcati `PENDING`, li preleva e li trasmette a RabbitMQ. Soltanto dopo aver ricevuto con certezza la ricevuta di ritorno (*Acknowledgement* o `ACK`) a livello TCP da RabbitMQ, il relay si volta verso il database MySQL aggiornando il flag dell'evento in `PROCESSED`, così che al giro successivo di polling venga ignorato. Questo processo assicura matematicamente la semantica di consegna *At-Least-Once* (Almeno una volta), un cardine dell'EDA. Nel peggiore dei casi (il relay crasha un nanosecondo dopo aver inviato il messaggio ma prima di marcare `PROCESSED`), l'evento verrà inviato due volte, circostanza tollerabile e arginabile dall'idempotenza a valle.

2.4 Il Pattern Saga e la Coreografia per i rimborsi

L'acquisto di un posto può non andare a buon fine lato utente, oppure la riserva temporanea su HoldMySeat (tipicamente di pochi minuti) può scadere fatalmente un istante prima che l'integrazione del pagamento giunga a conclusione. In queste circostanze di disallineamento temporale, è imperativo che il sistema provveda ad avviare un riaccredito dei fondi verso la carta del cliente, operazione nota come *Compensating Transaction* (transazione di compensazione).

In ambienti cloud, non è possibile applicare un Rollback globale, perché il primo pezzo del puzzle è già terminato ed ha prodotto esiti concreti esterni. Pertanto, si impiega il pattern **Saga**, implementato in questo progetto secondo lo stile della **Coreografia** (rispetto all'alternativa dell'Orchestrazione). Non esiste un supervisore centrale: ogni servizio genera un evento di errore al quale gli altri interessati reagiscono passivamente.

Il *Refund Worker* vive costantemente in ascolto asincrono sulle code RabbitMQ preposte alle cancellazioni (`refund.queue`). Al sopraggiungere di un evento, il worker emula un'invocazione di basso livello al provider esterno della banca. Poiché un server bancario non è sotto il nostro controllo, potrebbe rifiutare la chiamata con HTTP 500, impiegare troppo tempo a rispondere (timeout), o dropare i pacchetti. Se il tentativo fallisce, il worker solleva un'eccezione locale e restituisce deliberatamente un **NACK** (Negative Acknowledgement) al canale AMQP di RabbitMQ. RabbitMQ, istruito appositamente, comprenderà che il task è andato in fumo e provvederà a *riaccodare* (re-queue) celermente il messaggio per un tentativo successivo, in modo invisibile all'utente che ormai avrà già proseguito la sua navigazione. Questo realizza una coerenza eventuale molto solida.

Capitolo 3

L’Idempotenza e la gestione della Concorrenza

L’architettura distribuita descritta sopra introduce come compromesso essenziale la semantica ”At-Least-Once”. A sua volta, questa semantica obbliga tassativamente ogni nodo a supportare l’idempotenza. In questo capitolo svisceriamo i meccanismi software adottati in PayMySeat per difendersi dalla concorrenza dannosa.

3.1 Concetto teorico e utilità dell’Idempotenza

L’idempotenza è la proprietà per cui la reiterazione o moltiplicazione dell’esecuzione di una singola intenzione operativa non modifica lo stato finale oltre l’impronta generata dalla prima e singola esecuzione utile.

Nello standard HTTP REST, verbi quali GET, PUT, e DELETE sono contrattualmente assunti come idempotenti dal protocollo. Aggiornare un record con lo stesso valore per 100 volte genera sempre il medesimo record aggiornato e non altera il numero totale di record né incrementa valori. Al contrario, il verbo POST, largamente impiegato per la creazione di nuovi record (e usato universalmente nei gateway di checkout del carrello, *Create Payment*), non possiede questa proprietà. Se un client invia lo stesso body POST ripetutamente in un ciclo `while`, il backend creerà senza remore un nuovo addebito alla banca ad ogni girata di ciclo.

Il disastro si concretizza tipicamente quando la comunicazione va in timeout per packet loss sul nodo di ritorno (il client spedisce il payload, il server incassa, la banca manda l’ok al server, il server manda l’ok al client, ma la rete fa decadere l’ultimo miglio TCP). Il browser del client va in stallo e un utente apprensivo, osservando la rotellina in caricamento, compie l’azione più umana e deleteria possibile: ricarica la pagina per intero ignorando gli avvisi del browser e inviando due (o più) pacchetti POST identici e letali.

3.2 Primo scudo protettivo: La Cache Redis ad alte prestazioni

Per arginare chirurgicamente questo fenomeno, viene introdotto l’obbligo, per il frontend o per il chiamante, di specificare uno string header univoco, denominato `Idempotency-Key` (solitamente un UUIDv4 o un hash MD5 derivato dall’ordine). Il primissimo step operativo compiuto dal

controller di *Payment API* è instradare la chiave verso il demone ****Redis****, con il quale controlla un database in memoria RAM ad alte prestazioni:

- La API interroga Redis: `GET idempotency:a1b2c3d4`.
- Se la chiave **esiste già**, significa che stiamo affrontando un replay (volontario o involontario). L'API interrompe bruscamente il flusso, preleva il vecchio risultato JSON precedentemente salvato in RAM da Redis (la "memoria" del successo passato), e lo sputa verso il client senza accendere alcuna connessione verso il database primario e ignorando tutto.
- Se la chiave **non esiste**, la API prosegue in modo indisturbato con il processo pesante, contatta MySQL, crea gli eventi, e per ultimissima cosa, *salva l'esito* in Redis agganciandolo alla `Idempotency-Key`, impostando un TTL (Time-To-Live) utile di 24-48 ore per risparmiare preziosa RAM.

Il controllo su Redis richiede tra gli 0.2 e i 2 millisecondi. Di contro, processare l'intera transazione su disco con MySQL ne richiederebbe decine. In presenza di 10.000 utenti concorrenti, Redis fa da paraurti disperdendo il traffico DDoS indesiderato senza penalizzare lo spazio e i log.

3.3 Secondo scudo protettivo: I vincoli di unicità nel database Relazionale

Ma Redis presenta dei difetti congeniti da cui un ingegnere software deve sempre tutelarsi: non offre isolamento transazionale completo rispetto alle race-conditions estremamente ridotte (nell'ordine dei microsecondi) e può essere svuotato improvvisamente a causa di un crash e riavvio del demone, essendo un sistema volatile a meno di accorgimenti AOF (Append-Only File).

Se due macchine separate avviano due transazioni nel medesimo istante (poiché magari bilanciate round-robin dall'Ingress di Kubernetes) e leggono Redis in perfetta contemporaneità prima che la prima istanza abbia il tempo materiale di scrivervi l'esito, si verificherà la temibile Race Condition. Ambedue concluderanno che la chiave non esiste, e ambedue contatteranno allegramente MySQL scatenando il duplice addebito.

A proteggere irrevocabilmente i conti degli utenti vi è dunque il secondo livello, ovvero lo schema logico del DBMS Relazionale (RDBMS). Nel DDL di MySQL, la colonna `idempotency_key` possiede la qualifica rigorosa `UNIQUE CONSTRAINT`. Quando le due transazioni proveranno a eseguire un `INSERT`, il motore InnoDB di MySQL, che assicura internamente isolamento serializzabile sui blocchi e garanzie ACID globali, farà transitare la prima operazione bloccando e gettando nel cestino la seconda, lanciando un'eccezione insormontabile di tipo `IntegrityError`. L'applicazione software catturerà tale eccezione e saprà abortire l'esecuzione con un messaggio `HTTP 409 Conflict`, sanando il disastro prima che si manifesti.

Capitolo 4

Sicurezza Logica e Webhook Crittografici

Il confine asincrono fra PayMySeat e HoldMySeat è gestito dal paradigma dei Webhook. Anziché costringere il sistema di biglietteria a invocare periodicamente e faticosamente un endpoint per sapere lo status dell'ordine (approccio *Polling*, dispendioso per la rete ed erratico nella latenza), PayMySeat effettua chiamate Push in stile callback.

4.1 Il vettore d'attacco dello Spoofing

Il meccanismo dei Webhook è per sua natura passibile di gravi e grossolani attacchi informatici. Poiché il server HTTP ricevente di HoldMySeat è un URL pubblicamente instradabile su internet, chiunque disponga della url e formatti un payload JSON in modo plausibile potrebbe lanciare un banale attacco usando un tool come `curl`, iniettando pacchetti `POST` che recitano falsità come "Il carrello numero 53 è appena stato regolarmente saldato per l'importo di 200 euro". Se il ricevitore accettasse passivamente questi payload basandosi solo sul formato corretto, l'attaccante esfiltrerebbe biglietti VIP totalmente gratuiti a spese dell'organizzatore.

4.2 Prevenzione mediante firma crittografica HMAC

Per validare autenticità, provenienza e integrità del Webhook respingendo il *Payload Spoofing* o gli attacchi di tipo *Man In The Middle* (MitM), l'Adapter di Gateway implementa lo standard di sicurezza adottato da Stripe.

A tempo zero, fra le parti (o come variabile d'ambiente sicura `WEBHOOK_SECRET` nei segreti di infrastruttura) viene registrata e concordata una stringa alfanumerica di notevole lunghezza, lo *Shared Secret*. Questo segreto non viaggia mai lungo i cavi di rete e resta custodito nel Vault o in memorie segrete.

L'operazione svolta dall'Adapter, un attimo prima della chiamata HTTP al partner, segue i passi:

1. Serializza l'oggetto da inviare ottenendo la stringa esatta di testo che comporrà il body della `POST`.

2. Inizializza l'oggetto della funzione crittografica di hash **HMAC** utilizzando lo Shared Secret come chiave, il testo come messaggio base e il digest SHA-256 come algoritmo matematico unidirezionale (one-way).
3. Estrapola la rappresentazione esadecimale (hexdigest) lunga 64 caratteri prodotta dall'algoritmo.
4. Inserisce il digest calcolato all'interno di un custom header nella richiesta HTTP finale (ad esempio **X-PayMySeat-Signature**).

Dal lato opposto della barricata, il software di HoldMySeat estrarrà l'header e il corpo raw della richiesta (*senza formatarlo o alterarvi neppure uno spazio vuoto*). Procederà ricalcolando localmente in autonomia il digest HMAC-SHA256, usando la copia in suo possesso dello Shared Secret. L'uguaglianza rigorosa fra il digest calcolato e quello intercettato nell'header certifica tre cose in modo assolutamente matematico e irrevocabile:

- L'autore del messaggio conosce intimamente il segreto, pertanto l'autore può essere solo e unicamente PayMySeat.
- Il messaggio non ha subito la minima alterazione da parte di terzi intermediari, giacché anche l'aggiunta di un singolo virgola al payload provocherebbe uno smottamento a valanga stravolgendo interamente la digest string (effetto valanga tipico delle *hash functions*).

Qualsiasi payload generato con segreto mancante, scorretto o manomesso viene troncato severamente con lo status code **401 Unauthorized**.

Capitolo 5

Containerizzazione e Infrastruttura di Esecuzione

La transizione verso un ambiente Cloud non prescinde dalle tecnologie di astrazione dell'OS che favoriscono un deploy agile e svincolato dai colli di bottiglia del bare metal. Questo capitolo sviscera l'integrazione di Docker per l'approccio container-first e la conseguente orchestrazione logica dell'intero ecosistema.

5.1 Riproducibilità totale tramite Docker

Tutto il software compilato e sviluppato per PayMySeat è contenuto e pacchettizzato in container **Docker**. A nessun livello di deploy è consentito (o reso necessario) lanciare i comandi standard via terminale ospite per installare runtime Python, Node.js, versioni arcane del database o tool di supporto. La dipendenza esterna esclusiva si riduce alla sola esistenza del demone Docker sul nodo di calcolo. Questo paradigma si definisce *Immutable Infrastructure*.

Il confezionamento (build phase) è gestito minuziosamente all'interno dei file testuali denominati **Dockerfile**, ciascuno afferente al suo microservizio. Nella pipeline di creazione delle immagini sono state adottate le best practice globali relative sia alla sicurezza dell'immagine sia all'ottimizzazione del builder cache:

- Viene ereditata come root image la distruzione `python:3.12-slim`, limitando drasticamente i vettori d'attacco rispetto a versioni *fat*, grazie all'esclusione di software in eccesso o pacchetti di rete superflui.
- L'ordinamento logico dei comandi RUN, ADD e COPY è tutt'altro che casuale. Le istruzioni che cambiano meno di frequente sono impilate in cima. Le dipendenze di terze parti elencate in `requirements.txt` vengono copiate e subito installate (tramite `pip install`) **molto prima** dell'ingestione del codice sorgente di PayMySeat. L'ecosistema di build di Docker rileva che il file `requirements.txt` non subisce variazioni, e carica istantaneamente dalla memoria i binari pip precompilati in cache. Conseguentemente, quando un operatore o sviluppatore altera un banale file `.py` o aggiunge una rotta al framework Flask, il processo di deploy si limiterà ad accodare in millisecondi solo un infinitesimo layer aggiuntivo in fondo allo stack. Ciò che in un pattern monolitico male architettato avrebbe impiegato tra

i tre e i cinque minuti per reinstallare tutti i binari Python, si avvera ora nella vertiginosa frazione di un secondo.

5.2 Orchestrazione locale con Docker Compose

Prima di migrare a un orchestratore enterprise come Kubernetes su grande scala cloud, le relazioni e i flussi interni vengono cablati logicamente a livello infrastrutturale locale avvalendosi di `docker-compose.yml`. Esso unisce e intreccia 7 contenitori fisicamente isolati, costruendo bridge e reti logiche fittizie (`cloud-project-net`). Il grande vantaggio è la risoluzione dei nomi DNS *out-of-the-box*: un container `payment_api` si può semplicemente connettere a un `rabbitmq` senza mai doversi domandare quale sia l'indirizzo IP virtuale effettivo sottostante.

Il setup rivela il suo punto focale nella corretta sincronizzazione e gestione dell'ordine di accensione. Un servizio debole in architetture SOA cade qualora la sua base dati dovesse presentare interruzioni temporanee o rallentamenti in avvio. Per cui, la clausola `depends_on` introdotta nei descrittori di servizio per le APIs subordina severamente l'avvio alla condizione di salute `service_healthy` dei macro nodi (MySQL, Redis, RabbitMQ). Solo se i contenitori dichiarano all'engine di aver superato un tcp-ping sulla porta target, Docker sarà autorizzato a fare fuoco sui processi Python a valle, evitando cascate di fatali stack trace di tipo `ConnectionRefusedError`.

Capitolo 6

Infrastructure as Code e Distribuzione su AWS Cloud

Slegarsi dal paradigma on-premise è condizione indispensabile in architetture cloud-native. L'esigenza architetturale non consisteva soltanto nello spostare container funzionanti localmente verso la nuvola Amazon Web Services (AWS), bensì nel fare ciò impiegando una rigorosa, dichiarativa, e auditable stesura del codice, aderendo ciecamente all'astrazione dell'*Infrastructure as Code* (IaC).

6.1 La Pipeline di Provisioning: HashiCorp Terraform

La generazione fisica dell'hardware simulato, dello scaling e dei vincoli di routing sono dettati mediante codice dichiarativo scritto col linguaggio HCL di **Terraform**. Con Terraform non ci si cura minimamente di impartire una sequenza temporale di comandi operativi. L'ingegnere formula una planimetria finale di ciò che intende materializzare ed è compito del software interagire coi complessi alberi REST API di AWS e applicare i differenziali per colmare il divario fra "realtà" ed "intenzione".

La directory di progetto dedicata ai file Terraform modella specificamente:

- **VPC e Routing Privato:** Stabilire ed isolare una **Virtual Private Cloud** recante prefisso di instradamento `10.0.0.0/16`, provvista di tabelle di routing custom che disgreghino e distribuiscano le Subnet Pubbliche da agganciare ai Gateway di uscita globale.
- **Risorse EC2 Elastic Compute Cloud:** Unica concessione IaaS dell'architettura. Macchine base `t3.micro` destinate a fungere da nodi worker per l'alloggiamento del demone Docker. Le risorse non contengono e non devono mai contenere codice pre-installato: seguono il concetto di *Immutable Bootstrapping*.
- **Security Groups e Filtri Ingress/Egress:** La prima e principale barriera contro manipolazioni esterne al progetto. Il Security group dichiara che solo ed unicamente due porte di protocollo TCP possono essere colpite dall'esterno: la porta SSH 22 (riservata ad Ansible o manutenzione eccezionale) e la porta 5000 HTTP a vantaggio dei payload di transazione diretti al framework applicativo. Nient'altro transita da e per il mondo esterno.

Un artificio di particolare rilievo tecnologico introdotto nell'IaC è la **Generazione Dinamica delle Credenziali Crittografiche SSH**. L'approccio manuale prevede che si navighi a mano nella console AWS, si chieda un file `.pem` per l'infrastruttura e lo si inserisca nel codice. Al contrario, Terraform usa il provider `tls_private_key` per istruire l'ambiente di deploy locale a generare ex-novo un entropico paio asimmetrico di chiavi RSA a 4096-bit nel corso dello start-up, innalzando istantaneamente la porzione pubblica nel Vault di Amazon (`aws_key_pair`), e depositando temporaneamente e segretamente la chiave privata nel path dell'agente locale preposto all'orchestrazione software di lì a pochi istanti. L'intera catena di trust viene chiusa a tenuta stagna senza che file segreti permarchino impropriamente in commit storici sui repo GitHub.

6.2 La Pipeline di Configurazione software: Ansible

Dichiarata terminata con successo la fase di provisioning e disponendo ora del nascente IP pubblico della macchina virtuale, la catena di montaggio del *Continuous Deployment* sfocia e delega lo scettro ad **Ansible**.

Mentre Terraform edifica muri e stanze fisiche (IaaS), Ansible esegue mansioni di fine architettura strutturale e domotica su file OS (Configuration Management), servendosi unicamente delle chiavi effimere RSA prodotte poc'anzi. Non occorrono script daemon installati lato vittima (Ansible si vanta dello striscione "*Agent-less*"), serve unicamente possedere privilegi sudo e un tunnel crittografato aperto.

Il *Playbook* YAML applicato all'istanza EC2 remota ricalca concetti rigidamente idempotenti, un mantra iterativo secondo cui applicare le direttive tre, cento o mille volte produrrà il medesimo esito della prima occorrenza:

- Aggiunge formalmente ai trusted-repositories i server keys GPG di Docker per abilitare l'ingestione `apt` pulita.
- Ordina a Ubuntu l'installazione delle librerie del Docker Engine CE, del tool set Compose e dei Containerd.
- Dispone il trapianto del repository di sviluppo clonato (`scp/sync` delle directory) nelle aree *user-home* designate.
- Emette l'ultimo fatidico comando `docker compose up --build -d`.

Smarcatasi dai cruscotti utente di AWS, l'intera genesi intercontinentale di PayMySeat, a zero setup locale pregresso, è ridotta a non più di un paio di input digitati sulla shell d'inizio rigo.

Capitolo 7

Validazione Sperimentale e Sperimentazione Empirica

Un'architettura formalmente e astrattamente esatta può nascondere difetti strutturali colossali nel momento esatto in cui subentra l'entropia del carico utente. L'esercitazione empirica ha testato ed esasperato questi domini di guasto.

7.1 Verifica intensiva dell'Idempotenza API sotto Race Condition

A riprova del superamento della concorrenza deleteria del doppio-click, si è attinto alla standard library di programmazione multi-core Python per comporre lo script autoportante e distruttivo `demo.idempotenza.py`. Esso istanzia una batteria composta dal toolset `ThreadPoolExecutor` che imbraccia decine di Thread di sistema, armandoli tutti con l'esatto e sovrapponibile pacchetto POST provvisto per intero della medesima `Idempotency-Key` e direzionati simultaneamente, all'unisono verso i socket in listening dell'infrastruttura.

Evidenze di output analizzate nei log: Alla pressione del tasto di via, il sistema di orchestrazione accusa un modesto ma irrisorio carico di CPU ed elabora 10 richieste parallele in un arco misurabile di pochissimi frame macchina.

- **Caso Zero (il "Vincitore"):** Uno solo e unico tra tutti i process-thread prenderà materialmente possesso del varco. Il framework Python lo fa slittare per primo verso la cache Redis (che attesta virginità della transazione). Permesso accordato per interrogare ed impegnare intensamente le risorse del disco rigido MySQL. Viene redatto l'insert query della transazione contabile. Viene allocato l'evento su outbox. La API fa un respire sollievo e riempie in RAM il dump confermativo JSON prima di rimetterlo all'utente d'origine.
- **I Casi Concorrenti (i 9 "Respinti"):** Lo sciame ritardatario (parliamo di latenze sub-millisecondo tra le istruzioni processore) varca i cancelli. Trova dinanzi a sé un panorama alterato a vita: Redis innesca la lettura che rivela che lo *State* ha già subito alterazione. L'albero operativo del software recide il ramo verso MySQL ignorando a tutti gli effetti la direttiva ed estrude brutalmente in uscita lo scheletro della transazione JSON. Si manifesta in terminale HTTP Response 200 OK.

Il bilancio totale dell'esaminatore rivela che il Database è rimasto inalterato oltre l'unità, l'utente vede pagato unicamente il suo biglietto e i sistemisti gioiscono dell'impenetrabilità raggiunta e superata. La garanzia *Lock Ottimistico* è provata con elevata affidabilità.

7.2 Verifica estrema della Resilienza Outbox via Chaos Engineering

Simulare la caduta a catena di componenti infrastrutturali centrali è l'unico mezzo che distilla la resilienza in informatica. La branca di questa disciplina si identifica come *Chaos Engineering*. Con il supporto di macro direttive si è proceduto al Kill del processo di sistema RabbitMQ proprio nel frangente centrale della transazione dei pagamenti.

In una vecchia e mal architettata applicazione web, questo scatenerrebbe l'effetto domino bloccando i thread web e mandando in collasso l'API verso un disastroso HTTP 502/504 Timeout generalizzato.

Con PayMySeat la realtà registrata è abissalmente mutata.

- L'utente (il Frontend che naviga la dashboard) vede ugualmente il pop-up verde del successo. Il danaro è correttamente registrato. Nessuna eccezione visibile su user experience.
- Ad assorbire i danni in prima linea c'è il guardiano isolato *Outbox Relay*. Egli attinge l'evento e tenta coraggiosamente di contattare il nodo TCP del Broker (ora in down o distrutto). Trovando la porta sigillata, fallisce in via silente la routine asincrona loggando in background l'avvertimento, e con un sobrio passo indietro **rifiuta categoricamente** di etichettare la transazione come PROCESSED, preservandole a vita il delicato stato primordiale di PENDING nell'acciaio inossidabile del Database MySQL.
- Quindici minuti dopo la fine delle miserie umane, qualora un cron-job o Kubernetes provveda al ripristino di istanze container per RabbitMQ, l'*Outbox Relay* ristabilisce docilmente i tunnel comunicativi ed immette d'un lampo nella pipeline gli accumulati eventi PENDING, come niente fosse accaduto. Nessun addebito monetario si trasformerà in perdita permanente e incontrollata dell'account fruitore.

Capitolo 8

Conclusioni

A epilogo del progetto, la disamina meticolosa restituisce la fondatezza della teoria iniziale: PayMySeat risponde in modo robusto e scalabile orizzontalmente alle sfide complesse tipiche dei sistemi distribuiti che il Web di nuova concezione scarica in seno allo sviluppatore Cloud. L'abbattimento progressivo e consapevole delle *Fallacies of Distributed Computing* si realizza.

L'adozione ineccepibile del pattern di disaccoppiamento mediante Outbox Transazionale, combinata all'estetica isolata dei moduli asincroni ed alla capillarità della persistenza polivalente e poliglotta assicura di abbattere colli di bottiglia antichi di due decenni. Il progetto ha perimetrato il raggiungimento totale dei pilastri fondanti:

- **Correttezza Matematica e Finanziaria dei dati:** Idempotenza forte su due livelli indipendenti impedisce doppi addebiti letali per il business.
- **Resilienza Disaccoppiata:** Le dipendenze dei componenti possono incrinarsi e fallire ritardando flussi macro, ma i circuiti breaker consentono alla nave di riemergere dal caos applicando il *retry-loop*. Si ha il transito alla "Coerenza Eventuale".
- **Agilità ed Automazione:** I flussi Terraform ed Ansible codificano lo una pratica consolidata per un disaster-recovery e un deployment rapidissimo. L'ambiente è formalizzato a dovere in versionamenti Git e mai sottomesso al caos della configurazione puramente testuale.

L'esito complessivo forgia quindi in conclusione un vero e maturo prodotto "Cloud-Native", pronto per scenari gravosi.

.1 Evoluzione dalla Proposta al MVP (Minimum Viable Product)

Durante la fase di stesura iniziale (Progetto Teorico), l'architettura era stata ideata per includere nativamente l'utilizzo di servizi Cloud completamente gestiti su AWS, tra cui Amazon SQS/MQ in sostituzione a RabbitMQ, DynamoDB al posto di MongoDB, e l'adozione dell'orchestrazione tramite Kubernetes (EKS) con Auto-Scaling e bilanciatori ALB.

Ai fini della validazione sperimentale e per rientrare nei vincoli temporali del progetto, l'implementazione reale descritta nei capitoli successivi (e riproducibile nell'Appendice A) si basa

invece su un ambiente Docker Compose su macchine EC2, accompagnato da script Terraform e Ansible. Questa infrastruttura (IaaS) simula fedelmente le interazioni a microservizi, mentre l'architettura a servizi gestiti e Kubernetes delineata in proposta rappresenta la traiettoria di evoluzione per uno scenario produttivo definitivo (Future Work).

Appendice A

Manuale d'Uso e Replicazione delle Prove

Questa appendice contiene tutte le istruzioni necessarie per avviare l'infrastruttura in ambiente locale, eseguire la suite di test e validare empiricamente le garanzie architetturali di PayMySeat discusse nei capitoli precedenti. Seguendo passo passo questi comandi, l'esaminatore potrà constatare il corretto funzionamento dell'idempotenza, del pattern Transactional Outbox e delle comunicazioni asincrone con RabbitMQ.

A.1 Prerequisiti di Sistema

Per poter eseguire i test, la macchina host deve essere equipaggiata con il seguente software di base:

- **Docker e Docker Compose:** Fondamentali per l'orchestrazione locale e per sollevare il database relazionale (MySQL), il message broker (RabbitMQ) e la cache (Redis) senza inquinare l'host.
- **Python 3.12+:** Necessario per eseguire gli script di test concorrenti e le chiamate API simulate tramite le librerie standard.
- **Postman o cURL:** Utili per interrogare liberamente gli endpoint esposti dalle API in modalità interattiva.

A.2 Avvio dell'Infrastruttura

Aprire il terminale (o la PowerShell) e navigare all'interno della root del progetto. Prima di lanciare l'applicazione, assicurarsi che le porte di default (3306 per MySQL, 5672 per RabbitMQ, 6379 per Redis e 5000 per Flask) siano libere sull'host.

Per innescare il build delle immagini e l'avvio in background di tutti e sette i container, digitare:

```
1 docker compose up --build -d
```

L'operazione scaricherà le immagini ufficiali e compilerà i microservizi. Eseguire in seguito il seguente comando per monitorare che lo stato di tutti i servizi transiti su **Healthy**:

```
1 docker compose ps
```

In questo frangente, il file `init-db.sql` sarà già stato ingerito da MySQL, popolando lo schema con la tabella dei pagamenti e la tabella di outbox.

A.3 Prova 1: Flusso di Pagamento Standard (Happy Path)

Una volta che l'API è esposta sulla porta 5000, è possibile simulare un acquisto legittimo. L'endpoint in ascolto richiede un payload JSON e un header crittografico per l'idempotenza.

Eseguire la seguente invocazione tramite cURL:

```
1 curl -X POST http://localhost:5000/payments \
2   -H "Content-Type: application/json" \
3   -H "Idempotency-Key: test-happy-path-12345" \
4   -d '{"booking_id": "BK-001", "amount_cents": 2500, "currency": "EUR"}'
```

Comportamento atteso:

1. L'API risponderà quasi istantaneamente con un codice HTTP 200 OK e uno status PAID.
2. Osservando i log del relay (`docker compose logs -f outbox_relay`), si noterà l'intercettazione del messaggio PENDING e la sua pubblicazione verso RabbitMQ.
3. Osservando i log del gateway (`docker compose logs -f gateway_adapter`), si vedrà la ricezione del messaggio AMQP e il calcolo della firma HMAC, concludendo l'invio (simulato) al webhook esterno.

A.4 Prova 2: Test Concorrenziale di Idempotenza

L'obiettivo di questa prova è bombardare l'API con decine di richieste sovrapposte, tutte recanti lo stesso ID, per dimostrare che non vi sarà alcun doppio addebito e nessun crash del database per Constraint Violations, grazie al cuscinetto Redis.

Nella directory radice è presente lo script `demo_idempotenza.py`. Avviarlo tramite l'interprete:

```
1 python demo_idempotenza.py
```

Lo script utilizza la libreria `concurrent.futures.ThreadPoolExecutor` per spalancare 10 o più thread contemporanei che inoltreranno **esattamente la stessa POST** nello stesso millisecondo. **Comportamento atteso:** L'output a schermo certificherà che una (e una sola) risposta sarà elaborata ex-novo (elaborazione effettiva del pagamento e inserimento in DB), mentre le restanti $N-1$ richieste restituiranno il *medesimo JSON di successo*, ma contrassegnato logicamente come hit in cache da Redis, senza aver toccato MySQL la seconda volta. Interrogando poi il database manualmente (`SELECT * FROM payments`), ci sarà un unico record registrato.

A.5 Prova 3: Test di Resilienza (Chaos Engineering)

Questa simulazione convalida l'affidabilità dell'infrastruttura contro disservizi gravi. Il test si sviluppa uccidendo intenzionalmente il Broker dei Messaggi prima che l'API finisca di lavorare.

1. Dal terminale, arrestare forzatamente il container RabbitMQ:

```
1 docker compose stop rabbitmq
2
```

2. Ripetere una POST di pagamento (usando una nuova Idempotency Key, es. `test-chaos-001`).
3. L'API Web risponderà positivamente (200 OK): per il cliente il biglietto è salvo.
4. I log del demone *Outbox Relay* inizieranno a stampare eccezioni di tipo `AMQPConnectionError`, non potendo inoltrare l'evento. L'evento su DB rimarrà nel limbo temporaneo dello stato `PENDING`.
5. Riaccendere RabbitMQ per emulare il ripristino di rete:

```
1 docker compose start rabbitmq
2
```

6. Non appena il container rientra online (entro 5-10 secondi), il relay ristabilirà autonomamente il socket TCP, assorbirà l'evento rimasto incagliato nel DB e lo sparerà in coda, ripristinando la *Coerenza Eventuale* dell'intero sistema. Il ciclo di vita dell'ordine arriverà a compimento senza intervento umano.

A.6 Prova 4: Test di Compensazione (Saga Pattern)

In conformità al pattern Saga (Coreografia), è essenziale verificare il percorso di compensazione in caso di fallimento o rifiuto della transazione da parte del provider esterno. L'infrastruttura di test è stata programmata per simulare proattivamente un fallimento (`payment.failed`) per qualsiasi importo la cui cifra in centesimi termini per 13.

1. Inviare una richiesta di pagamento con un importo target, ad esempio 2013 centesimi:

```
1 curl -X POST http://localhost:5000/payments \
2     -H "Content-Type: application/json" \
3     -H "Idempotency-Key: test-saga-001" \
4     -d '{"booking_id": "BK-002", "amount_cents": 2013, "currency":
5     "EUR"}'
```

2. L'API processerà il pagamento, ma il *Gateway Adapter* rileverà l'importo terminante in 13.
3. **Comportamento atteso:** Osservando i log (`docker compose logs gateway_adapter`), si noterà la dicitura *"Importo terminante in 13. Simulazione fallimento"*. Il webhook

generato e inoltrato ad HoldMySeat non conterrà l'evento di successo, bensì un evento `payment.failed`. Questo innescherà nel sistema di biglietteria la **Compensating Transaction**, che rilascerà il lock sul posto a teatro rendendolo nuovamente acquistabile e chiudendo in maniera sicura il ciclo Saga.

A.7 Prova 5: Deploy in Cloud (AWS)

In questo step finale, si illustra l'utilizzo dei file IaC presenti nel progetto per eseguire il provisioning automatico su Amazon Web Services (AWS) e il setup dei container tramite Ansible. Questa prova richiede il possesso di credenziali AWS valide e di Terraform installato sulla macchina host.

1. Assicurarsi di aver configurato la CLI di AWS con chiavi di accesso dotate di privilegi EC2 e VPC:

```
1 aws configure
2
```

2. Navigare nella directory dell'infrastruttura Terraform:

```
1 cd infra/local/terraform
2
```

3. Inizializzare il workspace di Terraform, scaricando il provider AWS:

```
1 terraform init
2
```

4. Visionare l'Execution Plan, ovvero la preview formale di tutte le risorse IaaS che verranno create (reti VPC, macchine EC2, Security Groups e chiavi SSH):

```
1 terraform plan
2
```

5. Procedere al deploy effettivo dell'infrastruttura sul cloud. Il processo impiega tipicamente pochi minuti per istanziare i server:

```
1 terraform apply -auto-approve
2
```

6. Terminata la fase di IaaS, Terraform stamperà a terminale (negli *Outputs*) l'indirizzo IP pubblico della nuova istanza EC2 generata.

7. Tornare alla root del progetto ed eseguire il Configuration Management usando **Ansible**. Ansible utilizzerà la chiave SSH generata dinamicamente da Terraform per entrare nell'EC2, installarvi `docker` e `docker-compose`, copiare il progetto e avviare i container:

```
1 cd ../../../../
2 ansible-playbook -i "INDIRIZZO_IP_EC2," -u ubuntu --private-key ./infra/
  local/terraform/paymyseat-key.pem infra/local/ansible/playbook.yml
3
```

8. Adesso l'intero ecosistema PayMySeat è live su AWS. È possibile ripetere liberamente i test delle Prove 1, 2, 3 e 4 sostituendo `localhost` con il nuovo `INDIRIZZO_IP_EC2`.
9. Una volta conclusi i test, per evitare addebiti di fatturazione (billing) da parte di AWS, distruggere programmaticamente tutto l'ambiente creato con il singolo comando inverso:

```
1 cd infra/local/terraform
2 terraform destroy -auto-approve
3
```